Estrutura de Dados aplicada à otimização do KFKA

KFKA – Plataforma de Acompanhamento Escolar · Grupo NOT FOUND
Integrantes: Ana Julia Bernardo Quintans, Riane Toscano, Ricardo Branco Rios

1. Contexto

O KFKA registra o acompanhamento bimestral dos alunos (descrição, média e tags), que passa pela revisão do administrador antes de ser publicado aos responsáveis. Com o uso, o volume de alunos, acompanhamentos e registros de auditoria cresce a cada bimestre. Sem cuidado na escolha das estruturas de dados, telas de listagem, relatórios e exportações ficam lentas e consomem cada vez mais memória.

Este relatório mostra onde o grupo aplicou o conteúdo da disciplina para resolver esse problema, em duas partes do projeto:

- **Aplicação de listagem de alunos (C# / .NET 8 + SQLite)**: entrega de Estrutura de Dados que demonstra a paginação sobre uma lista.
- **Backend da plataforma (Java / Spring Boot)**: onde a mesma ideia foi levada para a aplicação real, junto com conjuntos e mapas.

2. Lista + paginação na listagem de alunos

Os alunos lidos do banco são guardados em uma List<Aluno>. A lista foi escolhida porque mantém a ordem da consulta (ORDER BY Id), cresce dinamicamente sem precisar saber o total antecipadamente e permite acesso direto por posição, o que é exatamente o que a paginação precisa.

Data/AlunoRepository.cs

```
1  List<Aluno> alunos = new();
2  comando.CommandText = "SELECT Id, Nome, Matricula, Turma FROM Alunos ORDER BY Id";
3  using var leitor = comando.ExecuteReader();
4  while (leitor.Read())
5  {
6      Aluno aluno = new() { Id = leitor.GetInt32(0), Nome = leitor.GetString(1),
7                            Matricula = leitor.GetString(2), Turma = leitor.GetString(3) };
8      alunos.Add(aluno);   // inserção no fim: O(1) amortizado
9  }
10 return alunos;
```

Figura 1 – Montagem da lista a partir do SQLite (Data/AlunoRepository.cs)

Com a lista pronta, o serviço de paginação calcula quantas páginas existem e recorta só os elementos da página pedida. A posição inicial é calculada diretamente — (página − 1) × tamanho — e só os k elementos da página são copiados para a tela, em vez de exibir os n registros de uma vez.

Services/PaginacaoService.cs

```cs
public int CalcularTotalPaginas(List<Aluno> alunos, int tamanhoPagina)
    => (int)Math.Ceiling((double)alunos.Count / tamanhoPagina);

public List<Aluno> ObterPagina(List<Aluno> alunos, int paginaAtual, int tamanhoPagina)
{
    if (paginaAtual < 1 || paginaAtual > CalcularTotalPaginas(alunos, tamanhoPagina))
        throw new ArgumentOutOfRangeException(nameof(paginaAtual), "Página inexistente.");
    return alunos.Skip((paginaAtual - 1) * tamanhoPagina)
                 .Take(tamanhoPagina)
                 .ToList();
}
```

Figura 2 – Cálculo e recorte da página (Services/PaginacaoService.cs)

Resultado: com 20 alunos e 5 por página, o usuário navega por 4 páginas (`N`, `A`, `P`) e a validação impede o acesso a páginas inexistentes. A aplicação é didática: os dados ficam todos em memória, e por isso a consulta SQL propositalmente não usa LIMIT/OFFSET.

3. Paginação levada ao backend

Na plataforma real, manter todos os registros em uma lista na memória do servidor não escala. A otimização foi mover o recorte da página para o banco de dados: as listagens recebem um Pageable (número da página, tamanho e ordenação) e devolvem um Page, de modo que o servidor só carrega k registros por requisição. O mesmo raciocínio da Seção 2 foi aplicado, mas a memória usada deixa de depender do total de registros.

controller/AlunoController.java

```java
@GetMapping
public ResponseEntity<Page<Aluno>> listar(@RequestParam(defaultValue="") String nome,
                                          Pageable pageable) {
    return ResponseEntity.ok(service.listarAluno(nome, pageable));
}
```

Figura 3 – Listagem paginada (controller/AlunoController.java)

A exportação de relatórios em CSV usa a mesma estrutura para percorrer os dados em lotes de 200 registros: cada página é processada, escrita no arquivo e descartada antes de buscar a próxima.

service/RelatorioService.java

```java
Pageable pagina = PageRequest.of(0, 200, Sort.by("id"));
Page<AcompanhamentoResposta> resultado;
do {
    resultado = consultar(filtro, pagina);
    for (AcompanhamentoResposta a : resultado.getContent()) { /* escreve a linha */ }
    pagina = resultado.nextPageable();
} while (resultado.hasNext());
```

Figura 4 – Exportação CSV em lotes (service/RelatorioService.java)

4. Conjuntos (Set) para evitar duplicatas e buscar rápido

Um acompanhamento pode ter várias tags, e um responsável pode estar vinculado a vários alunos. Nos dois casos, repetir o mesmo item não faz sentido. Por isso os identificadores chegam na requisição como Set<Long> e as entidades são guardadas em HashSet: se o front-end enviar [1, 1, 2], o conjunto fica apenas com {1, 2}, sem nenhum código extra de verificação. No banco, a tabela acompanhamento_tag ainda tem restrição de unicidade como segunda proteção.

service/AcompanhamentoService.java

```java
private Set<Tag> buscarTags(Set<Long> ids) {
    Set<Tag> selecionadas = new HashSet<>();
    for (Long id : ids) {
        Tag tag = tags.findById(id)
            .orElseThrow(() -> erro(HttpStatus.NOT_FOUND, "Tag não encontrada"));
        if (!tag.getAtivo()) throw erro(HttpStatus.BAD_REQUEST, "Tag deve estar ativa");
        selecionadas.add(tag);
    }
    return selecionadas;
}
```

Figura 5 – Tags sem repetição (service/AcompanhamentoService.java)

O filtro de auditoria é executado em toda requisição HTTP e precisa decidir se o método altera dados. Em vez de uma cadeia de comparações, usa-se um conjunto imutável: a verificação contains é feita por hash, em tempo constante em média.

security/AuditoriaFilter.java

```java
private static final Set<String> MUTACOES = Set.of("POST", "PUT", "PATCH", "DELETE");
...
if (MUTACOES.contains(req.getMethod()) && ...)
    service.registrar(usuario, req.getMethod(), req.getRequestURI(), res.getStatus());
```

Figura 6 – Pertencimento em O(1) (security/AuditoriaFilter.java)

5. Mapa (Map) para agrupar os indicadores

O painel de indicadores mostra a média de cada disciplina. A forma ingênua seria, para cada disciplina, percorrer todos os acompanhamentos filtrando os dela — custo O(n × d), em que d é o número de disciplinas. Com groupingBy, os registros são distribuídos em um Map<Disciplina, List<Acompanhamento>> em uma única passada O(n), e a média de cada grupo é calculada só sobre os seus próprios elementos.

service/RelatorioService.java

```java
Map<Disciplina, List<Acompanhamento>> grupos = registros.stream()
    .collect(Collectors.groupingBy(Acompanhamento::getDisciplina));

List<IndicadoresResposta.MediaDisciplina> disciplinas = grupos.entrySet().stream()
    .map(entry -> new IndicadoresResposta.MediaDisciplina(entry.getKey().getId(),
         entry.getKey().getNome(), mediaDe(entry.getValue())))   // média do grupo
    .sorted(Comparator.comparing(IndicadoresResposta.MediaDisciplina::disciplina))
    .toList();
```

Figura 7 – Agrupamento por disciplina (service/RelatorioService.java)

(Trecho simplificado: no código original, a média do grupo é calculada em linha com `reduce` e `divide`).

6. Resumo

Estrutura	Onde foi usada	Problema resolvido	Custo da operação principal
Lista (List<Aluno>)	ListagemAlunos (C#) – AlunoRepository e PaginacaoService	Guardar os alunos lidos do banco em ordem e recortar uma página	Inserção O(1) amortizado; acesso por índice O(1); uma página custa O(k), k = tamanho da página
Página (Page / Pageable)	Backend Java – todas as listagens e exportação CSV	Evitar carregar todos os registros na memória do servidor	Memória O(k) por requisição em vez de O(n)
Conjunto (HashSet / Set)	Tags do acompanhamento, alunos do responsável, filtro de auditoria	Impedir duplicatas e verificar pertencimento rapidamente	add / contains O(1) em média
Mapa (Map via groupingBy)	Indicadores do RelatorioService	Calcular a média por disciplina	Uma passada O(n) em vez de O(n × d)

7. Conclusão e próximos passos

A escolha das estruturas seguiu a operação mais frequente em cada caso: lista quando a ordem e o acesso por posição importam, página quando o volume de dados cresce sem limite, conjunto quando não pode haver repetição e a pergunta é “este item está aqui?”, e mapa quando é preciso agrupar por uma chave. Com isso, as listagens têm custo proporcional ao tamanho da página, as relações não aceitam itens duplicados e os indicadores são calculados em uma única passada.

Pontos de melhoria identificados durante a análise:

- A exportação XLSX e os indicadores ainda carregam todos os registros filtrados em uma lista; podem passar a ler em lotes, como já faz o CSV.
- A busca de tags faz uma consulta por id; trocar por `findAllById(ids)` reduz N consultas a apenas uma.